

Millores d'eficiència en programes recursius i iteratius

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Politècnica Superior d'Enginyeria
de Vilanova i la Geltrú

Índex

- 1 Eficiència o cost d'un programa
- 2 Millores d'eficiència de programes recursius
- 3 Exercicis (millores d'eficiència de programes recursius)
- 4 Millores d'eficiència de programes iteratius
- 5 Exercicis (millores d'eficiència de programes iteratius)

Índex

- 1 Eficiència o cost d'un programa
- 2 Milllores d'eficiència de programes recursius
- 3 Exercicis (milllores d'eficiència de programes recursius)
- 4 Milllores d'eficiència de programes iteratius
- 5 Exercicis (milllores d'eficiència de programes iteratius)

Eficiència o cost d'un programa (I)

Eficiència d'un programa:

Mesura dels recursos (temps de CPU i espai de memòria) que requereix per executar-se.

- Cost del temps: Estimació del temps que trigarà el programa en executar-se, en funció del temps necessari per realitzar cadascuna de les operacions elementals.

Eficiència o cost d'un programa (I)

Eficiència d'un programa:

Mesura dels recursos (temps de CPU i espai de memòria) que requereix per executar-se.

- Cost del temps: Estimació del temps que trigarà el programa en executar-se, en funció del temps necessari per realitzar cadascuna de les operacions elementals.
- Cost de l'espai: Estimació de la mesura de l'espai que ocupa a memòria durant la seva execució; seria la suma de les mides de totes les variables que utilitza.

Eficiència o cost d'un programa (II)

- L'eficiència pot ser utilitzada com a criteri de comparació de diversos programes que resolen un mateix problema.

Eficiència o cost d'un programa (II)

- L'eficiència pot ser utilitzada com a criteri de comparació de diversos programes que resolen un mateix problema.
- Per ara ens centrarem només en el cost temporal.

Consideracions

- No es tenen en compte factors externs al programa: màquina on s'executa, sistema operatiu o llenguatge de programació.

Consideracions

- No es tenen en compte factors externs al programa: màquina on s'executa, sistema operatiu o llenguatge de programació.
- Mesurem el temps d'execució en termes del nombre d'instruccions del programa en relació amb la mida de les dades (dimensió dels vectors, valor dels nombres, dimensió de les matrius. . .).

Consideracions

- No es tenen en compte factors externs al programa: màquina on s'executa, sistema operatiu o llenguatge de programació.
- Mesurem el temps d'execució en termes del nombre d'instruccions del programa en relació amb la mida de les dades (dimensió dels vectors, valor dels nombres, dimensió de les matrius. . .).
- Només considerarem el cost del cas pitjor.

Exemples (I)

- Cost **constant**: Obtenir la mida d'un vector, pila, cua o llista (amb `size()`); obtenir el valor d'una posició d'un vector, el cim d'una pila, el primer d'una cua, l'arrel d'un arbre o l'element referenciat per un iterador d'una llista.

Exemples (I)

- Cost **constant**: Obtenir la mida d'un vector, pila, cua o llista (amb `size()`); obtenir el valor d'una posició d'un vector, el cim d'una pila, el primer d'una cua, l'arrel d'un arbre o l'element referenciat per un iterador d'una llista.
- Cost **lineal**: Suma dels elements d'una pila, un vector, etc.; cerques simples; assignació o còpia de vectors, llistes, etc. de tipus simples.

Exemples (I)

- Cost **constant**: Obtenir la mida d'un vector, pila, cua o llista (amb `size()`); obtenir el valor d'una posició d'un vector, el cim d'una pila, el primer d'una cua, l'arrel d'un arbre o l'element referenciat per un iterador d'una llista.
- Cost **lineal**: Suma dels elements d'una pila, un vector, etc.; cerques simples; assignació o còpia de vectors, llistes, etc. de tipus simples.
- Cost **quadràtic**: Algorismes simples d'ordenació de vectors (inserció, bombolla...), veure si una estructura seqüencial té tots els seus elements diferents (sense ordenar-los).

Exemples (II)

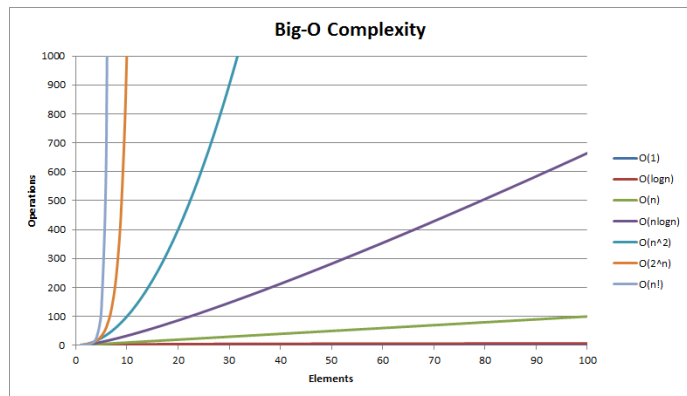
- Cost **logarítmic**: Cerca dicotòmica, algorismes ràpids pel càlcul de la potència o pel càlcul de l'arrel quadrada.

Exemples (II)

- Cost **logarítmic**: Cerca dicotòmica, algorismes ràpids pel càlcul de la potència o pel càlcul de l'arrel quadrada.
- Cost **exponencial**: Generació de combinacions, permutacions, etc., de n elements; algorismes de cerca exhaustiva de candidats de mida n per complir una propietat (p.e. tractar de descobrir per força bruta una contrasenya).

Exemples (III)

La figura següent mostra set funcions típiques en l'anàlisi d'algorismes sobre diferents mides del problema. Indiquen el temps que trigarà a executar-se un algorisme (operació) en funció de la mida de les dades.



[Font: Big O Notation and Complexity]

Índex

1 Eficiència o cost d'un programa

2 **Milllores d'eficiència de programes recursius**

3 Exercicis (milllores d'eficiència de programes recursius)

4 Milllores d'eficiència de programes iteratius

5 Exercicis (milllores d'eficiència de programes iteratius)

Milllores d'eficiència de programes recursius

- Partirem de programes recursius correctes i intentarem estalviar càlculs innecessaris.

Milllores d'eficiència de programes recursius

- Partirem de programes recursius correctes i intentarem estalviar càlculs innecessaris.
- Eina: la immersió

Milllores d'eficiència de programes recursius

- Partirem de programes recursius correctes i intentarem estalviar càlculs innecessaris.
- Eina: la immersió
 - **Immersió d'eficiència per dades:** Afegir paràmetres que passin informació d'una crida a la següent per no haver-la de tornar a calcular-la.

Milllores d'eficiència de programes recursius

- Partirem de programes recursius correctes i intentarem estalviar càlculs innecessaris.
- Eina: la immersió
 - **Immersió d'eficiència per dades:** Afegir paràmetres que passin informació d'una crida a la següent per no haver-la de tornar a calcular-la.
 - **Immersió d'eficiència per resultats:** Obtenir informació procedent de la següent crida recursiva per estalviar-nos instruccions a l'actual.

Exemple: la successió de Fibonacci (I)

Es defineix com:

$$fib(n) = \begin{cases} 0, & \text{si } n = 0 \\ 1, & \text{si } n = 1 \\ fib(n-1)+fib(n-2), & \text{si } n \geq 2 \end{cases}$$

Valors:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 ...

Exemple: la successió de Fibonacci (II)

Implementació:

```
int fib (int n)
/* Pre: n >= 0 */
/* Post: El resultat és el terme n-èssim de la
    successió de Fibonacci */
{
    int res;
    if (n < 2)
        res = n;
    else
        res = fib(n-1) + fib(n-2);
    return res;
}
```

Exemple: la successió de Fibonacci (II)

Implementació:

```
int fib (int n)
/* Pre: n >= 0 */
/* Post: El resultat és el terme n-èssim de la
    successió de Fibonacci */
{
    int res;
    if (n < 2)
        res = n;
    else
        res = fib(n-1) + fib(n-2);
    return res;
}
```

Quins càlculs es repeteixen? Si $n = 20$, quines crides es fan? Quines crides es fan quan $n = 19$? Solució?

Exemple: la successió de Fibonacci (III)

Ens interessa una **immersió per resultats**: Una crida **tornarà dos valors** consecutius de la successió.

Exemple: la successió de Fibonacci (III)

Ens interessa una **immersió per resultats**: Una crida **tornarà dos valors** consecutius de la successió.

```
pair<int, int> fib_ef (int n)
/* Pre: n > 0 */
/* Post: El primer component del resultat és fib(n);
   el segon és fib(n-1) */
{
    pair<int, int> y;
    if (n == 1)
        y = make_pair(1, 0);
    else {
        y = fib_ef(n-1);
        /* Hl: y.first és fib(n-1); y.second és fib(n-2) */
        /* Fita: n */
        y = make_pair(y.first + y.second, y.first);
    }
    return y;
}
```

Exemple: la successió de Fibonacci (IV)

Implementació de la funció original:

```
int fib (int n)
/* Pre: n >= 0 */
/* Post: El resultat és el terme n-èssim de la
successió de Fibonacci */
{
    int res;
    pair<int, int> res2;
    if (n == 0)
        res = 0;
    else {
        res2 = fib_ef(n);
        res = res2.first; // descartem el resultat adicional
    }
    return res;
}
```

Exemple: la successió de Fibonacci (IV)

Implementació de la funció original:

```
int fib (int n)
/* Pre: n >= 0 */
/* Post: El resultat és el terme n-èssim de la
   successió de Fibonacci */
{
    int res;
    pair<int, int> res2;
    if (n == 0)
        res = 0;
    else {
        res2 = fib_ef(n);
        res = res2.first; // descartem el resultat adicional
    }
    return res;
}
```

El cost ha passat d'exponencial a lineal perquè per calcular `fib(n)` cridem a `fib_ef(n)` i aquesta produeix `n` crides (que tenen un nombre constant d'operacions). ✓

Exemple: la successió de Fibonacci (V)

Provem una solució alternativa amb **immersió per dades**: Una crida **passarà a la següent** dos valors consecutius de la successió. Per saber quan parar, cal un paràmetre extra.

Exemple: la successió de Fibonacci (V)

Provem una solució alternativa amb **immersió per dades**: Una crida **passarà a la següent** dos valors consecutius de la successió. Per saber quan parar, cal un paràmetre extra.

```
int fib_ef_param(int n, int i, int f1, int f2)
/* Pre: 0 < i <= n, f1 = fib(i-1), f2 = fib(i) */
/* Post: El resultat és el terme n-èssim de la
    successió de Fibonacci */
{
    int f;
    if (i == n)
        f = f2;
    else
        f = fib_ef_param(n, i+1, f2, f1+f2);
    /* HI: Post */
    /* Fita: n - i */
    return f;
}
```

Exemple: la successió de Fibonacci (V)

Provem una solució alternativa amb **immersió per dades**: Una crida **passarà a la següent** dos valors consecutius de la successió. Per saber quan parar, cal un paràmetre extra.

```
int fib_ef_param(int n, int i, int f1, int f2)
/* Pre: 0 < i <= n, f1 = fib(i-1), f2 = fib(i) */
/* Post: El resultat és el terme n-èssim de la
   successió de Fibonacci */
{
    int f;
    if (i == n)
        f = f2;
    else
        f = fib_ef_param(n, i+1, f2, f1+f2);
        /* HI: Post */
        /* Fita: n - i */
    return f;
}
```

Recursivitat lineal? I final?

Exemple: la successió de Fibonacci (VI)

Implementació de la funció original:

```
int fib (int n)
/* Pre: n >= 0 */
/* Post: El resultat és el terme n-èssim de la
    successió de Fibonacci */
{
    int res;
    if (n == 0)
        res = 0;
    else
        res = fib_ef_param (n, 1, 0, 1);
    return res;
}
```


Exemple: la successió de Fibonacci (VI)

Implementació de la funció original:

```
int fib (int n)
/* Pre: n >= 0 */
/* Post: El resultat és el terme n-èssim de la
    successió de Fibonacci */
{
    int res;
    if (n == 0)
        res = 0;
    else
        res = fib_ef_param (n, 1, 0, 1);
    return res;
}
```

El cost també és lineal. ✓

Índex

- 1 Eficiència o cost d'un programa
- 2 Millores d'eficiència de programes recursius
- 3 Exercicis (millores d'eficiència de programes recursius)**
- 4 Millores d'eficiència de programes iteratius
- 5 Exercicis (millores d'eficiència de programes iteratius)

Exercicis (eficiència de programes recursius)

- ❶ Diem que un element d'una pila de nombres és *dominador* si és més gran o igual que la suma dels elements més antics (visualment, els que se situen més avall) que ell. Donada una pila no buida de naturals, escriu una funció que obtingui el seu element dominador més recent o, equivalentment, més alt.

```
int dom(stack<int> p)
/* Pre: p = P, P no està buida i està formada per naturals */
/* Post: El resultat és el dominador més alt de P */
```

- ❷ Un arbre binari es diu *balancejat* si és buit o bé si els seus dos fills són balancejats i el valor absolut de la diferència de les seves altures no és més gran que 1. Dissenya una funció recursiva que estalviï càlculs innecessaris.

```
bool balancejat(const arbreBin<int> &a)
/* Pre: a = A */
/* Post: El resultat indica si A és un arbre balancejat */
```

Índex

- 1 Eficiència o cost d'un programa
- 2 Millores d'eficiència de programes recursius
- 3 Exercicis (millores d'eficiència de programes recursius)
- 4 Millores d'eficiència de programes iteratius**
- 5 Exercicis (millores d'eficiència de programes iteratius)

Millores d'eficiència de programes iteratius

- Partirem de programes iteratius correctes i intentarem estalviar càlculs innecessaris.

Millores d'eficiència de programes iteratius

- Partirem de programes iteratius correctes i intentarem estalviar càlculs innecessaris.
- Estratègia: Fer servir el resultat de cada iteració per minimitzar el cost del càlcul de la següent. El valor d'aquestes variables ha de constar a l'invariant.

Exemple: sèrie de Taylor de la funció e^x (I)

Es defineix com:

$$e^x = 1 + x + \frac{x^2}{2!} + \dots + \frac{x^n}{n!}$$

Idea inicial: Iterar la suma terme a terme sense aprofitar els termes anteriors, fent servir funcions auxiliars per calcular la potència (x^i) i el factorial ($i!$).

Exemple: sèrie de Taylor de la funció e^x (II)

```
double taylor_exp(double x, int n)
/* Pre: n >= 0 */
/* Post: El resultat és la suma fins n de la
   sèrie de Taylor per a e^x */
{
    double t;
    if (x == 0)
        t = 1;
    else {
        t = 1;
        int i = 1;
        /* Inv: t és la suma fins i-1 de la sèrie de Taylor per a e^x */
        while (i <= n) {
            t += pot(x, i) / fact(i);
            ++i;
        }
    }
    return t;
}
```


Exemple: sèrie de Taylor de la funció e^x (III)

Especificació de les dues funcions auxiliars:

```
double pot(double x, int n);  
/* Pre: n >= 0; x != 0 si n = 0 */  
/* Post: El resultat és x^n */  
}  
  
int fact(int n);  
/* Pre: n >= 0 */  
/* Post: El resultat és n! */  
}
```

Exemple: sèrie de Taylor de la funció e^x (IV)

- A cada iteració repetim el càlcul de la potència des de 0. Però sabem que

$$\text{pot}(x, i) = x * \text{pot}(x, i-1)$$

Exemple: sèrie de Taylor de la funció e^x (IV)

- A cada iteració repetim el càlcul de la potència des de 0. Però sabem que

$$\text{pot}(x, i) = x * \text{pot}(x, i-1)$$

- A cada iteració repetim el càlcul del factorial des de 0. Però sabem que

$$\text{fact}(i) = i * \text{fact}(i-1)$$

Exemple: sèrie de Taylor de la funció e^x (IV)

- A cada iteració repetim el càlcul de la potència des de 0. Però sabem que

$$\text{pot}(x, i) = x * \text{pot}(x, i-1)$$

- A cada iteració repetim el càlcul del factorial des de 0. Però sabem que

$$\text{fact}(i) = i * \text{fact}(i-1)$$

- Solució: no cal repetir els càlculs a cada iteració si els guardem en dues variables i els anem actualitzant.

Exemple: sèrie de Taylor de la funció e^x (V)

```
double taylor_exp_ef1(double x, int n)
/* Pre: n >= 0 */
/* Post: El resultat és la suma fins n de la s. de Taylor per a e^x */
{
    double t;
    if (x == 0) t = 1;
    else {
        t = 1;
        int i = 1;
        int f = 1;
        double p = 1;
        /* Inv: t és la suma fins i-1 de la sèrie de Taylor per a e^x;
           p = x^(i-1); f = (i-1)! */
        while (i <= n) {
            p = p * x;
            f = f * i;
            t += p / f;
            ++i;
        }
    }
    return t;
}
```

Exemple: sèrie de Taylor de la funció e^x (VI)

```
double taylor_exp_ef2(double x, int n)
/* Pre: n >= 0 */
/* Post: El resultat és la suma fins n de la s. de Taylor per a e^x */
{
    double t;
    if (x == 0)
        t = 1;
    else {
        t = 1;
        int i = 1;
        double pf = 1; // una única variable per contenir el nou terme
        /* Inv: t és la suma fins i-1 de la sèrie de Taylor per a e^x;
           pf = x^(i-1)/(i-1)! */
        while (i <= n) {
            pf = pf * x;
            pf = pf / i;
            t += pf;
            ++i;
        }
    }
    return t;
}
```

Índex

- 1 Eficiència o cost d'un programa
- 2 Millores d'eficiència de programes recursius
- 3 Exercicis (millores d'eficiència de programes recursius)
- 4 Millores d'eficiència de programes iteratius
- 5 Exercicis (millores d'eficiència de programes iteratius)

Exercicis d'eficiència de programes iteratius (I)

- 1 Escriu una funció que calculi de manera eficient el $\cos(x)$ fent servir el seu desenvolupament de Taylor al punt zero fins al terme n -èssim, és a dir:

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots + (-1)^n \frac{x^{2n}}{(2n)!}$$

- 2 Diem que un element d'una llista de nombres és dominador si és més gran o igual que la suma dels elements anteriors (els que se situen més a prop del seu `begin`) que ell. Donada una llista no buida de naturals, escriu una funció que obtingui el seu element dominador més avançat, és a dir, el més allunyat del `begin`.

```
int dom(const list<int> &f)
/* Pre: f no està buida i està formada per naturals */
/* Post: El resultat és el dominador més avançat de f */
```


Exercicis d'eficiència de programes iteratius (II)

- 3 Diem que un vector d'enters està mitjanament ordenat si cada element, tret del primer, és més gran o igual que la mitja dels elements anteriors a ell. Donat un vector no buit d'enters, escriu una funció que comprovi si el vector està mitjanament ordenat.

```
bool mitj_ordenat(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat indica si v està mitjanament ordenat */
```